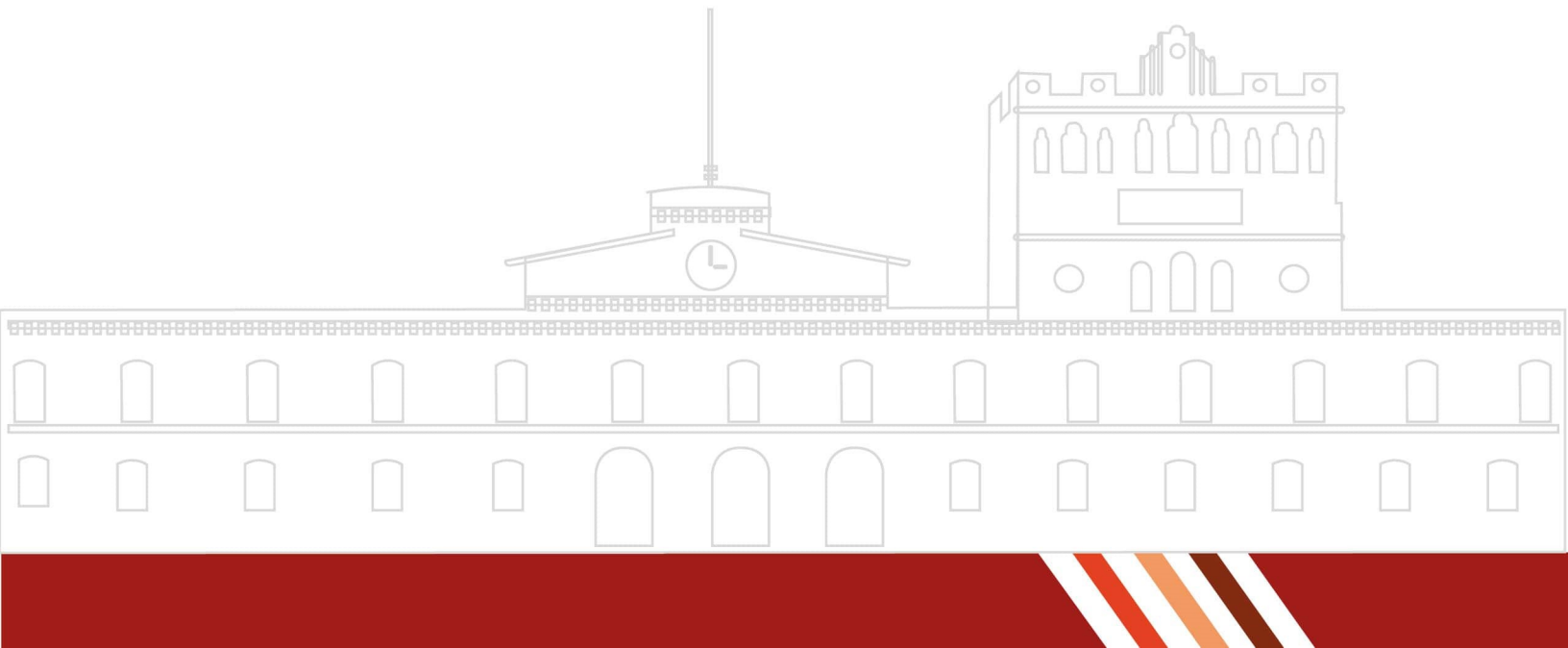


REPORTE DE PRÁCTICA FINAL

3.5 Práctica Final. Análisis sintáctico de expresiones de SQL

ALUMNO: Arrazola Ibarra Juan Pablo
Caballero Rossano Gabriel
Olivares Roque Diego
Vazquez Pontaza Fernando

Dr. Eduardo Cornejo-Velázquez



Introducción

En la actualidad, los sistemas gestores de bases de datos utilizan el lenguaje SQL como principal medio para realizar consultas, modificaciones y administración de información. Debido a la importancia de este lenguaje en el desarrollo de software y manejo de datos, resulta fundamental comprender cómo se interpretan y validan sus instrucciones de manera interna.

En esta práctica se desarrolló un analizador sintáctico de expresiones SQL utilizando las herramientas Flex y Bison. El proyecto tuvo como finalidad reconocer y validar la estructura gramatical de distintas instrucciones SQL mediante el uso de reglas léxicas y sintácticas previamente definidas.

El analizador implementado es capaz de procesar consultas como SELECT, INSERT, UPDATE y DELETE, además de incluir soporte para instrucciones complementarias como CREATE TABLE, DROP TABLE e instrucciones con condiciones lógicas y operaciones JOIN. Para ello, se diseñó una gramática que permite verificar que cada consulta cumpla con la sintaxis correcta del lenguaje SQL.

El sistema funciona leyendo un archivo de texto que contiene diferentes consultas SQL. Posteriormente, el analizador léxico identifica los tokens presentes en las instrucciones y el analizador sintáctico verifica si la estructura cumple con las reglas establecidas. Finalmente, el programa muestra mensajes indicando si cada consulta es válida o si contiene errores sintácticos.

Con esta práctica fue posible comprender de mejor manera el funcionamiento interno de los compiladores y analizadores de lenguaje, así como reforzar conocimientos relacionados con gramáticas, procesamiento de cadenas y validación sintáctica.

Marco Teórico

Compiladores

Un compilador es un programa encargado de traducir un lenguaje de alto nivel a un lenguaje que pueda ser entendido por la computadora. Durante este proceso se realizan diferentes etapas que permiten analizar, validar y transformar el código fuente. Entre las fases más importantes se encuentran el análisis léxico, el análisis sintáctico y el análisis semántico.

Los compiladores utilizan reglas gramaticales y estructuras formales para verificar que las instrucciones escritas por el usuario cumplan con la sintaxis establecida por el lenguaje de programación.

Análisis Léxico

El análisis léxico es la primera fase de un compilador. Su función principal es leer el código fuente carácter por carácter para identificar patrones y agruparlos en unidades llamadas tokens.

Los tokens representan elementos básicos del lenguaje, como palabras reservadas, identificadores, operadores, símbolos y constantes. El analizador léxico también se encarga de ignorar espacios en blanco, saltos de línea y comentarios que no afectan la estructura del programa.

En esta práctica se utilizó Flex como herramienta para construir el analizador léxico encargado de reconocer instrucciones y símbolos pertenecientes al lenguaje SQL.

Tokens

Los tokens son las unidades mínimas reconocidas durante el análisis léxico. Cada token representa un componente específico del lenguaje y permite que el analizador sintáctico pueda interpretar correctamente la estructura de las instrucciones.

Algunos ejemplos de tokens utilizados en esta práctica fueron:

- SELECT
- INSERT
- UPDATE
- DELETE
- FROM
- WHERE
- ID
- NUM
- CADENA

Análisis Sintáctico

El análisis sintáctico es la etapa encargada de verificar que la secuencia de tokens obtenida durante el análisis léxico cumpla con las reglas gramaticales definidas para el lenguaje.

Esta fase utiliza una gramática formal para determinar si las instrucciones están correctamente estructuradas. Si una consulta no cumple con la gramática establecida, el analizador genera un error sintáctico.

En esta práctica se utilizó Bison para implementar el analizador sintáctico y definir las reglas correspondientes a las consultas SQL.

Gramáticas

Una gramática es un conjunto de reglas que define cómo deben construirse las instrucciones de un lenguaje. Las gramáticas permiten establecer el orden correcto de los elementos que forman una sentencia.

Las reglas gramaticales utilizadas en el proyecto permitieron validar instrucciones SQL como SELECT, INSERT, UPDATE y DELETE, además de operaciones adicionales como CREATE TABLE, DROP TABLE y consultas con condiciones lógicas.

SQL

SQL (Structured Query Language) es un lenguaje utilizado para administrar y manipular bases de datos relacionales. Permite realizar operaciones como consultas, inserciones, modificaciones y eliminación de información almacenada en tablas.

Entre las instrucciones SQL utilizadas en esta práctica se encuentran:

- SELECT: utilizada para consultar información.
- INSERT: utilizada para insertar registros.
- UPDATE: utilizada para modificar datos existentes.
- DELETE: utilizada para eliminar registros.
- CREATE TABLE: utilizada para crear tablas.
- DROP TABLE: utilizada para eliminar tablas.

Flex

Flex es una herramienta utilizada para generar analizadores léxicos automáticamente a partir de expresiones regulares. Su función es identificar patrones dentro de un archivo de entrada y devolver tokens que posteriormente serán utilizados por el analizador sintáctico.

En esta práctica Flex permitió reconocer palabras reservadas, operadores, identificadores, cadenas y números pertenecientes al lenguaje SQL.

Bison

Bison es una herramienta utilizada para generar analizadores sintácticos a partir de gramáticas formales. Permite definir reglas de producción y validar que la secuencia de tokens recibida cumpla con la estructura establecida.

El archivo generado con Bison contiene las reglas gramaticales necesarias para analizar las consultas SQL implementadas en el proyecto.

Objetivos

Objetivo General

Desarrollar un analizador sintáctico de expresiones SQL utilizando las herramientas Flex y Bison, capaz de reconocer y validar consultas mediante reglas léxicas y gramaticales previamente definidas.

Objetivos Específicos

- Implementar un analizador léxico para identificar tokens pertenecientes al lenguaje SQL.
- Diseñar una gramática que permita validar la estructura de instrucciones SQL.
- Reconocer consultas SQL como SELECT, INSERT, UPDATE y DELETE.
- Detectar errores sintácticos en consultas mal estructuradas.
- Utilizar Flex y Bison para automatizar el proceso de análisis léxico y sintáctico.
- Procesar archivos de texto que contengan múltiples consultas SQL.
- Comprender el funcionamiento básico de un compilador y sus diferentes etapas de análisis.

Desarrollo

Análisis de Requisitos

Para el desarrollo del analizador sintáctico se establecieron inicialmente los requisitos funcionales del sistema. El programa debía ser capaz de reconocer y validar distintas instrucciones del lenguaje SQL mediante reglas léxicas y sintácticas.

Entre los principales requisitos definidos se encuentran los siguientes:

- Reconocer instrucciones SQL como SELECT, INSERT, UPDATE y DELETE.
- Validar la estructura correcta de las consultas.
- Detectar errores sintácticos en instrucciones mal construidas.
- Permitir la lectura de múltiples consultas desde un archivo de texto.
- Mostrar mensajes indicando si la consulta es válida o incorrecta.
- Implementar el sistema utilizando Flex, Bison y lenguaje C.

Diseño del Analizador Léxico

El analizador léxico fue desarrollado utilizando Flex. Su función consiste en identificar los componentes básicos del lenguaje SQL mediante expresiones regulares.

Dentro del archivo léxico se definieron palabras reservadas, identificadores, operadores, cadenas y símbolos especiales utilizados en las consultas SQL.

En la Tabla 1 se muestran algunos de los tokens implementados en el analizador léxico.

Token	Descripción
SELECT	Consulta de datos
INSERT	Inserción de registros
UPDATE	Actualización de datos
DELETE	Eliminación de registros
FROM	Selección de tabla
WHERE	Condición lógica
ID	Identificador
NUM	Valores numéricos
CADENA	Cadenas de texto

Tabla 1: Tokens implementados en el analizador léxico.

Diseño del Analizador Sintáctico

El analizador sintáctico fue implementado utilizando Bison. En esta etapa se definieron las reglas gramaticales necesarias para validar la estructura de las consultas SQL.

La gramática implementada permite reconocer instrucciones como:

- SELECT
- INSERT INTO
- UPDATE SET
- DELETE FROM
- CREATE TABLE
- DROP TABLE

- INNER JOIN

Además, se agregaron reglas para operadores lógicos AND y OR, así como expresiones relacionales.

Estructura General del Sistema

El sistema funciona mediante la interacción entre el analizador léxico y el analizador sintáctico. Primero, Flex procesa el archivo de entrada y convierte las palabras y símbolos en tokens. Posteriormente, Bison recibe dichos tokens y verifica si cumplen con la gramática definida. El funcionamiento general del sistema se muestra en la Figura 1.

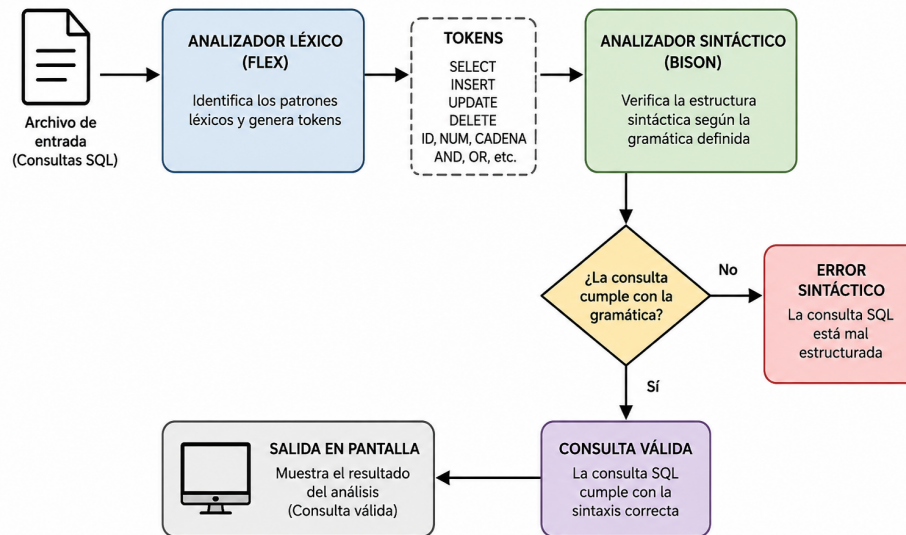


Figura 1: Funcionamiento general del analizador SQL.

Lectura de Archivos

El programa fue diseñado para recibir un archivo de texto como parámetro de entrada. El archivo contiene múltiples consultas SQL que posteriormente son procesadas por el sistema.

La lectura del archivo se realiza mediante lenguaje C utilizando la variable `yyin`, la cual permite conectar el archivo con el analizador generado por Flex y Bison.

Manejo de Errores

Se implementó una función de manejo de errores llamada `yyerror()`, cuya finalidad es mostrar mensajes cuando una consulta SQL presenta errores sintácticos.

Gracias a esta función el programa puede continuar analizando el resto de las consultas presentes en el archivo incluso después de detectar errores.

Compilación y Ejecución

Para generar el ejecutable del proyecto se utilizaron los comandos de Flex, Bison y GCC. El proceso de compilación consiste en:

1. Generar el analizador léxico con Flex.
2. Generar el parser utilizando Bison.

3. Compilar los archivos generados mediante GCC.
4. Ejecutar el programa enviando un archivo SQL como entrada.

El sistema muestra mensajes indicando si cada consulta es válida o contiene errores sintácticos.

Resultados

Para comprobar el correcto funcionamiento del analizador sintáctico se realizaron diversas pruebas utilizando consultas SQL válidas e inválidas.

El programa recibió como entrada un archivo de texto que contenía múltiples instrucciones SQL. Entre las consultas utilizadas se incluyeron instrucciones SELECT, INSERT, UPDATE, DELETE y DROP TABLE. En la Figura 2 se muestra el archivo de entrada utilizado para realizar las pruebas del sistema.

```
SELECT * FROM usuarios;
SELECT id, nombre, edad FROM clientes WHERE edad > 18;
INSERT INTO empleados VALUES (101, 'Carlos', 2500.50);
UPDATE inventario SET cantidad = 50, estado = 'A' WHERE id = 12;
DELETE FROM registros WHERE fecha < '2023-01-01' AND status = 'inactivo';
DROP TABLE users;
|
SELECT nombre FROM alumnos WHERE
UPDATE empleados SET salario = 1000
DELETE FROM tabla WHERE id >
```

Figura 2: Archivo de entrada con consultas SQL válidas e inválidas.

Posteriormente, el analizador procesó cada una de las consultas y mostró mensajes indicando si la estructura sintáctica era correcta o si existían errores dentro de la instrucción.

Las consultas válidas fueron reconocidas correctamente por el sistema, mientras que las instrucciones incompletas o mal estructuradas generaron mensajes de error sintáctico.

En la Figura 3 se muestra la salida generada por el programa durante el análisis del archivo de entrada.

```

C:\Users\gaboc\Downloads\AYC\SQL>compilador ejemplo.txt
=====
ANALIZANDO SCRIPT SQL: ejemplo.txt
=====

--- SELECT ---
-> CONSULTA VALIDA

--- SELECT ---
-> CONSULTA VALIDA

--- INSERT ---
-> CONSULTA VALIDA

--- UPDATE ---
-> CONSULTA VALIDA

--- DELETE ---
-> CONSULTA VALIDA

--- DROP TABLE ---
-> CONSULTA VALIDA

>>> ERROR SINTACTICO: La consulta SQL esta mal estructurada <<<

--- UPDATE ---
>>> ERROR SINTACTICO: La consulta SQL esta mal estructurada <<<

>>> ERROR SINTACTICO: La consulta SQL esta mal estructurada <<<

=====
ANALISIS FINALIZADO
=====

```

Figura 3: Resultado del análisis sintáctico realizado por el programa.

Durante las pruebas realizadas se observó que el analizador fue capaz de:

- Reconocer correctamente consultas SQL válidas.
- Detectar errores sintácticos en instrucciones incompletas.
- Procesar múltiples consultas dentro de un mismo archivo.
- Mostrar mensajes claros indicando el resultado del análisis.

Los resultados obtenidos demuestran que el analizador sintáctico implementado funciona correctamente y cumple con los objetivos planteados en la práctica.

Conclusiones

Durante el desarrollo de esta práctica fue posible comprender de manera más profunda el funcionamiento de los analizadores léxicos y sintácticos, así como la forma en que los compiladores procesan y validan un lenguaje formal mediante reglas gramaticales.

Uno de los principales aprendizajes obtenidos fue el uso de Flex y Bison para la construcción de un analizador SQL funcional. Se fortalecieron habilidades relacionadas con el diseño de gramáticas, reconocimiento de tokens, manejo de expresiones regulares y validación sintáctica de instrucciones.

También se adquirieron conocimientos sobre la estructura del lenguaje SQL y la manera en que las consultas pueden interpretarse internamente mediante procesos automáticos de análisis. Además, se reforzó el uso del lenguaje C para integrar el analizador y permitir la lectura de archivos de entrada.

Entre las habilidades fortalecidas destacan:

- Diseño e implementación de gramáticas.
- Uso de Flex para el análisis léxico.
- Uso de Bison para el análisis sintáctico.
- Detección y manejo de errores sintácticos.
- Integración de herramientas de compilación con lenguaje C.
- Procesamiento de archivos con múltiples consultas SQL.

Uno de los momentos más interesantes de la práctica fue observar cómo el sistema era capaz de identificar automáticamente consultas válidas e inválidas únicamente mediante las reglas definidas en la gramática. Resultó sorprendente comprobar cómo herramientas como Flex y Bison permiten construir analizadores similares a los utilizados en compiladores reales.

Asimismo, algunas dificultades surgieron durante el manejo de errores sintácticos y la correcta definición de las reglas gramaticales, especialmente al trabajar con múltiples tipos de instrucciones SQL y operadores lógicos. Sin embargo, estas situaciones permitieron comprender mejor el comportamiento interno del parser y mejorar la lógica del proyecto.

Finalmente, la práctica permitió relacionar los conceptos teóricos vistos en clase con una implementación práctica y funcional, fortaleciendo conocimientos fundamentales sobre compiladores, análisis de lenguajes y procesamiento sintáctico.

Referencias Bibliográficas

References

- [1] Grabowska, S.; Saniuk, S. (**2022**). Business models in the industry 4.0 environment—results of web of science bibliometric analysis. *J. Open Innov. Technol. Mark. Complex*, 8(1), 19.
- [2] Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (**2008**). *Compiladores: Principios, técnicas y herramientas* (2da ed.). Pearson Educación.
- [3] Levine, J. R. (**2009**). *Flex & Bison*. O'Reilly Media.